

1 Proeminența Culmilor Montane

1.1 Introducere

Se dau N altitudini $a_1, a_2, \dots, a_N$ reprezentând profilul unui lanț muntos. Pentru fiecare vârf (punct care este mai înalt decât vecinii săi), trebuie calculată *proeminența* — diferența dintre înălțimea vârfului și cea mai adâncă vale situată între el și următorul vârf mai înalt, fie că acesta se află în stânga sau în dreapta sa.

Proeminența unui vârf P se definește ca:

$$\text{prom}(P) = \min_{(Q \text{ mai înalt decât } P)} (H_Q - \min_{\text{vale între } P \text{ și } Q} H_{\text{vale}})$$

unde Q este primul vârf mai înalt decât P în orice direcție.

1.2 Observații Cheie

1. Un vârf este definit formal: $a_{i-1} < a_i > a_{i+1}$. Este suficient să considerăm doar vârfurile, nu și văile.
2. **Eliminarea duplicatelor:** Valorile identice consecutive sunt ignorate, deoarece nu influențează proeminența.
3. **Cazul $C = 1$:** Se cere doar numărul vârfurilor. Răspunsul este pur și simplu numărul de indici i cu $1 < i < N$ care satisfac condiția de vârf.
4. **Problema centrală ($C = 2$):** Pentru fiecare vârf, trebuie determinată proeminența față de cel mai apropiat vârf mai înalt.

1.3 Soluția cu Stiva

Ideea fundamentală: pentru fiecare vârf, trebuie găsit primul vârf mai înalt din stânga și cea mai adâncă vale dintre ele. Aceasta se poate rezolva în timp liniar folosind o stivă care menține vârfurile în ordine strict descrescătoare a înălțimii.

Algoritmul (parcurea stânga-dreapta):

1. Parcurgem vectorul de la stânga la dreapta.
2. Când întâlnim un vârf nou $a[i]$:
 - El devine candidat pentru viitoare vârfuri.
 - Eliminăm din stivă toate vârfurile cu înălțime $\leq a[i]$ — acestea nu mai pot fi primul vârf mai înalt pentru niciun vârf viitor.
 - Dacă stiva nu este goală, vârful din vârf stivei (cel mai apropiat mai înalt) determină proeminența stânga: $\text{prom}[i] = a[i] - \text{valley}[\text{top}]$.
 - În caz contrar, $a[i]$ este un vârf absolut (mai înalt decât toate cele din stânga), deci proeminența stânga este chiar $a[i]$.
3. **Mentinerea văii:** Variabila $\text{valley}[\text{top}]$ stochează cea mai joasă vale întâlnită de la vârful curent din stivă până la poziția curentă i .

1.4 Mirarea Vectorului

Pentru a calcula proeminența și spre dreapta, soluția oglindește vectorul (inversează ordinea) și repetă același algoritm. Rezultatul pentru vârful original de pe poziția i va fi în $\text{prom}[1][N - 1 - i]$ după a doua parcurgere.

Proeminența finală a unui vârf este minimum dintre proeminența stânga și proeminența dreapta:

$$\text{prominenta}(i) = \min(\text{prom}[0][i], \text{prom}[1][N - 1 - i])$$

1.5 Corectitudine

Lema 1 (Menținerea stivei): La orice moment, vârfurile din stivă sunt în ordine strict descrescătoare a înălțimii și în ordine crescătoare a poziției.

Demonstrație: Un vârf nou este adăugat doar după ce toate vârfurile mai mici sau egale au fost eliminate. Deci ultimul vârf din stivă are întotdeauna o înălțime strict mai mare decât cele eliminate, iar poziția sa este mai mare (fiindcă parcurgerea este de la stânga la dreapta). $\square$

Lema 2 (Cea mai adâncă vale): Pentru vârful din vârful stivei P și vârful curent V , $\text{valley}[\text{top}]$ reprezintă cea mai joasă vale dintre P și V .

Demonstrație: La fiecare pas, dacă $a[i]$ este mai mică decât $\text{valley}[\text{top}]$, actualizăm $\text{valley}[\text{top}]$. Astfel, această variabilă stochează minimumul tuturor valorilor întâlnite între P și V . $\square$

Lema 3 (Proeminența stânga): După procesarea poziției i , $\text{prom}[0][i]$ conține proeminența stânga corectă a vârfului de pe poziția i .

Demonstrație: Din Lema 1, vârful din vârful stivei este primul vârf mai înalt din stânga lui i . Din Lema 2, $\text{valley}[\text{top}]$ este cea mai adâncă vale între aceștia. Diferența lor este exact proeminența stânga. $\square$

1.6 Complexitate

- **Timp:** $O(N)$ — fiecare element este adăugat și eliminat din stivă cel mult o dată.
- **Memorie:** $O(N)$ — stocăm vectorul și stiva.

1.7 I

Implementare în C++

```
#define NMax 1000005
long N, C, s;
long a[NMax];

long peak[NMax], valley[NMax], top = -1;
long prom[2][NMax];

void popTop() {
```

```

    if (top < 0) return;
    if (top >= 1 && valley[top-1] > valley[top]) {
        valley[top-1] = valley[top];
    }
    top--;
}

int main() {
    freopen("proeminenta.in", "r", stdin);
    freopen("proeminenta.out", "w", stdout);

    scanf("%ld_%ld", &C, &N);
    for (int i = 0; i < N; i++) {
        scanf("%ld", &a[i]);
        if (i > 0 && a[i] == a[i-1]) {
            i--; N--;
        }
    }
    if (C == 1) {
        for (int i = 1; i < N-1; i++) {
            if (a[i-1] < a[i] && a[i] > a[i+1]) s++;
        }
        printf("%ld\n", s);
        return 0;
    }

    for (int d = 0; d <= 1; d++) {
        for (int i = 1; i < N-1; i++) {
            if (a[i-1] < a[i] && a[i] > a[i+1]) {
                prom[d][i] = a[i];
                while (top >= 0 && a[peak[top]] <= a[i])
                    popTop();
                if (top >= 0 && a[peak[top]] > a[i])
                    prom[d][i] = a[i] - valley[top];
                peak[++top] = i;
                valley[top] = a[i];
            }
            if (top >= 0 && valley[top] > a[i]) valley[top]
                = a[i];
        }
        // Oglindim vectorul pentru parcurgerea dreapta-
        // stanga
        for (int i = 0; i < N / 2; i++) {
            long aux = a[i]; a[i] = a[N-1-i]; a[N-1-i] = aux;
        }
        top = -1;
    }
    // Combin m rezultatele din cele dou directii
    for (int i = 1; i < N-1; i++)

```

```

        if (a[i-1] < a[i] && a[i] > a[i+1])
            if (prom[0][i] > prom[1][N-1-i])
                prom[0][i] = prom[1][N-1-i];

    for (int i = 1; i < N-1; i++)
        if (prom[0][i] > 0) printf("%ld_", prom[0][i]);
    printf("\n");
    return 0;
}

```

1.8 Concluzii

Soluția folosește elegant o stivă monotonică pentru a rezolva problema proeminenței în timp liniar. Ideea cheie este că, la întâlnirea unui vârf mai înalt, toate vârfurile mai mici devin irelevante pentru viitoarele calcule — ele sunt eliminate în timp $O(1)$ amortizat. Mirarea vectorului transformă problema bilaterală în două probleme unidimensionale identice.